UPN, PASIÓN POR TRANSFORMAR VIDAS

ANALISIS DE ALGORITMOS Y ESTRATEGIAS DE PROGRAMACION

SESIÓN 9: BackTracking

Ing. Mitchell Blancas

LOGRO DE LA SESIÓN



Al término de la sesión el estudiante aplica Backtracking en la solución de problemas propuestos y los representa a través de algoritmos.

¡Pregunta!

¿En que casos se aplica backtracking?



	6		4	7			2	5
	7	4		1		8		3
1					6	7	9	
	9	5		3	2	4		
7		2			9			1
			5		7	2	8	
6		1	3	5				
			7			6	1	8
4	8			9		3		



SUDOKU



Sudoku

sū = número, doku = solo



Es un tablero de **9x9** compuesto por regiones cuadradas de **3x3**. Algunas celdas ya tienen números, el objetivo es rellenar las celdas vacías, con un número de tal forma que cada columna, fila y región contengan los números del 1 al 9 sin que se repitan.

5	2		3		4		6	
1	8	4	6		9		3	
	6	9		7	8	4		
7	4			3		1		5
		1	2	9	7		4	
9	3	8		4	5		7	
		5	9				8	
4	9			8	1	2	5	
			4					3

Un Sudoku bien planteado debe tener una única solución.

Sudoku

```
#include <iostream>
using namespace std;
int sudoku[9][9]= {
  {6,0,4,0,5,0,2,0,8},
  {0,5,0,0,0,0,0,4,0},
  {1,0,0,0,0,0,0,0,9},
  {5,6,8,0,9,0,4,3,2},
  {4,0,0,8,0,6,0,0,5},
  {7,0,0,5,2,4,0,0,1},
  {0,0,5,0,8,0,9,0,0},
  {0,4,2,0,0,0,6,8,0},
  {8,0,6,2,0,9,5,0,3}
};
void resSudoku(int, int);
bool buscaCeldaVacia(int &, int &);
bool puedoColocar(int, int, int);
void impSudoku(void);
int main()  {
   resSudoku(0,0);
   return 0;
}
```

```
void resSudoku(int x, int y) {
   bool hayVacia=buscaCeldaVacia(x, y);
   if ( !hayVacia )  impSudoku();
   else {
        for(int nro=1; nro<=9; nro++)
            if ( puedoColocar(x, y, nro) ) {
               sudoku[x][y]=nro;      // lo coloco
               resSudoku(x,y+1);    // tomo el sgte
               sudoku[x][y]=0;        // lo quito
            }
    }
}

bool buscaCeldaVacia(int &x, int &y) {
   for( ; x<9; x++) {
        for(  ; y<9; y++)
             if( sudoku[x][y]==0 )
                return true;
        y=0;
    }
   return false;
}
```

Sudoku

```
bool puedoColocar(int f, int c, int nro) {
    int i, j;
    // chequeo fila
    for(j=0; j<9; j++)
        if(sudoku[f][j]==nro)
                return false;

    // chequeo columna
    for(i=0; i<9; i++)
        if(sudoku[i][c]==nro)
                return false;

    // chequeo subcuadrado
   int iniCol = (c/3) *3;
   int finCol = iniCol+3;
   int iniFil  = (f/3) *3;
   int finFil  = iniFil+3;

   for(i=iniFil; i<finFil; i++)
       for(j=iniCol; j<finCol; j++)
                if(sudoku[i][j]==nro)
                  return false;

   return true;
}
```

```
void impSudoku(void) {
   int i, j;
   cout<<"--SOLUCION--"<<endl;
   for(i=0; i<9; i++)  {
       for(j=0; j<9; j++) {
                cout<<sudoku[i][j];
                if ( j==2 ||  j==5 )
                    cout<<" ";
       }
       cout<<endl;
       if ( i==2 || i==5 )
            cout<<" "<<endl;
   }
   cout<<endl;
}
```

Sudoku

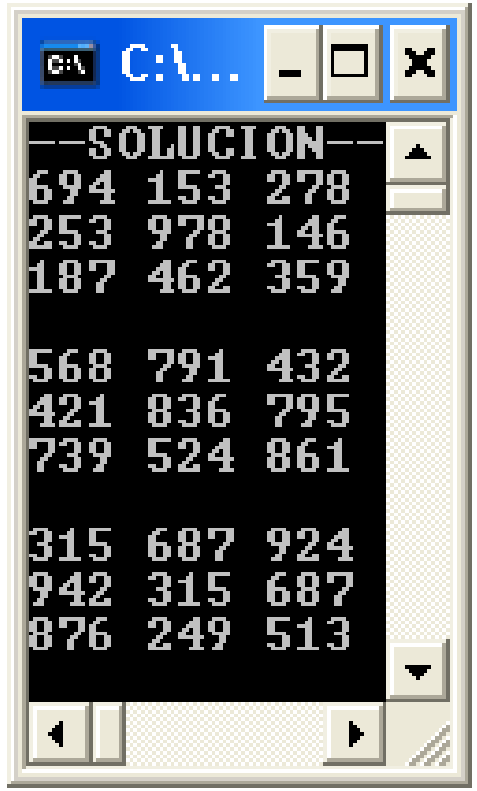
Entrada:

```
604 050 208
050 000 040
100 000 009

568 090 432
400 806 005
700 524 001

005 080 900
042 000 680
806 209 503
```

Salida:



El Principio del minimax

Suponga que se tienen 2 jugadores



MAXI Rodríguez



MINNIE Mouse

MAXI y **MINNIE** juegan cada uno un turno uno detrás de otro. Si el juego se encuentra en un estado cualquiera, y le toca jugar a **MAXI**, él querrá maximizar su posibilidad de ganar, para elegir su movimiento piensa en el mejor movimiento que haría **MINNIE** y actúa en consecuencia.

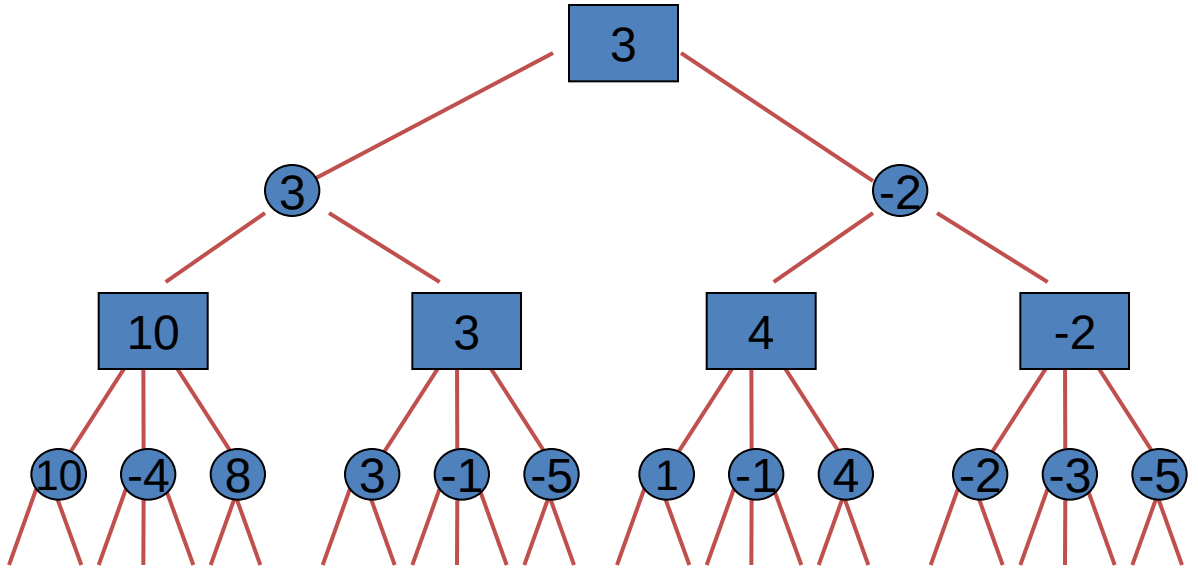
Turno de:











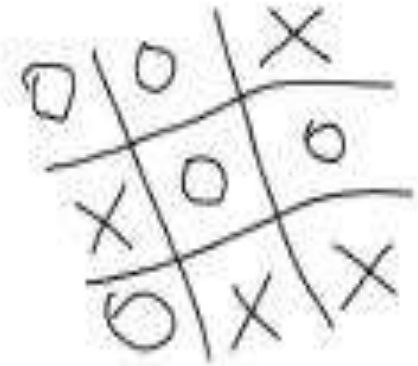
Para asignar valores a los nodos más bajos se usa alguna función de evaluación que nos indique que tan bueno es ese estado para MAXI, si es el turno de MINNIE ella tratará de minimizar ese valor, si es el turno de MAXI, él tratará de maximizar ese valor.

3 en Raya

Escriba un programa que permita jugar "3 en raya" contra la computadora







```
#include <iostream>
using namespace std;
#define GRANDE 100

#define MAX 1
#define MIN   2

#define NADIE               '-'
#define JUGADOR          'O'
#define COMPUTADOR 'X'

void impTablero(void);
int bt(int );
int eval(void);

char t[][3]=
{   {NADIE,NADIE,NADIE},
    {NADIE,NADIE,NADIE},
    {NADIE,NADIE,NADIE}
};
int mejorFila, mejorCol;
```

```
int main()
{  int f, c, ganador;
   impTablero();
   do
   {
       do
       {    cout<<"Su Turno:"<<endl;
            cout<<"Fila? ";    cin>>f;
            cout<<"Columna? "; cin>>c;
        } while( (f<0 || f>=3 || c<0 || c>=3 ) || t[f][c] != NADIE);

       t[f][c]= JUGADOR;
       impTablero();
       bt (MAX);
       cout<<"Computadora:"<<endl;
       cout<<"Fila: "<<mejorFila<<endl;
       cout<<"Columna: "<<mejorCol<<endl;
       t[mejorFila][mejorCol]= COMPUTADOR;
       impTablero();
       ganador =eval();
   } while(ganador==2);

   if(ganador==-1) cout<<"***Ud. Gana***"l;
   else if(ganador==1) cout<<"***Computadora gana***“;
   else cout<<"***Empate!***“;
   cout<<endl;
}
```

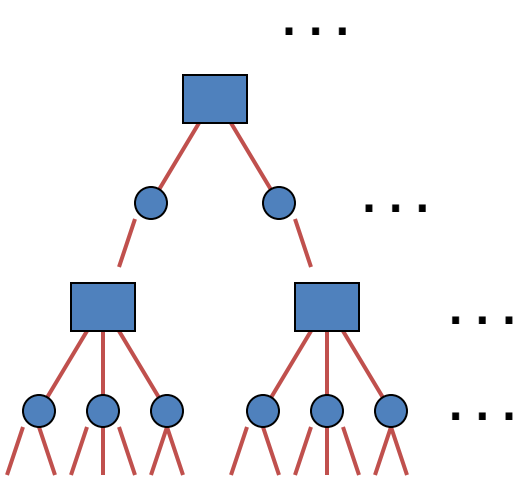


```
int bt(int modo)
{   int i, j, val, mfLocal, mcLocal;
    int valor= eval();
    if( valor!=2 ) return valor;

    if ( modo==MAX ) valor=-GRANDE;
    else valor=GRANDE;

    for (i=0; i<3; i++)
        for (j=0; j<3; j++)
            if ( t[i][j]==NADIE )
            {   if ( modo==MAX )
                {   t[i][j]=COMPUTADOR;
                    val= bt ( MIN );
                    if ( valor<val ) { valor=val; mfLocal=i; mcLocal=j; }
                }
                else
                {   t[i][j]=JUGADOR;
                    val= bt( MAX );
                    if( valor>val ) {  valor=val; mfLocal=i;  mcLocal=j;  }
                }
                t[i][j]=NADIE;
            }
   mejorFila=mfLocal;
   mejorCol=mcLocal;
   return valor;
}
```

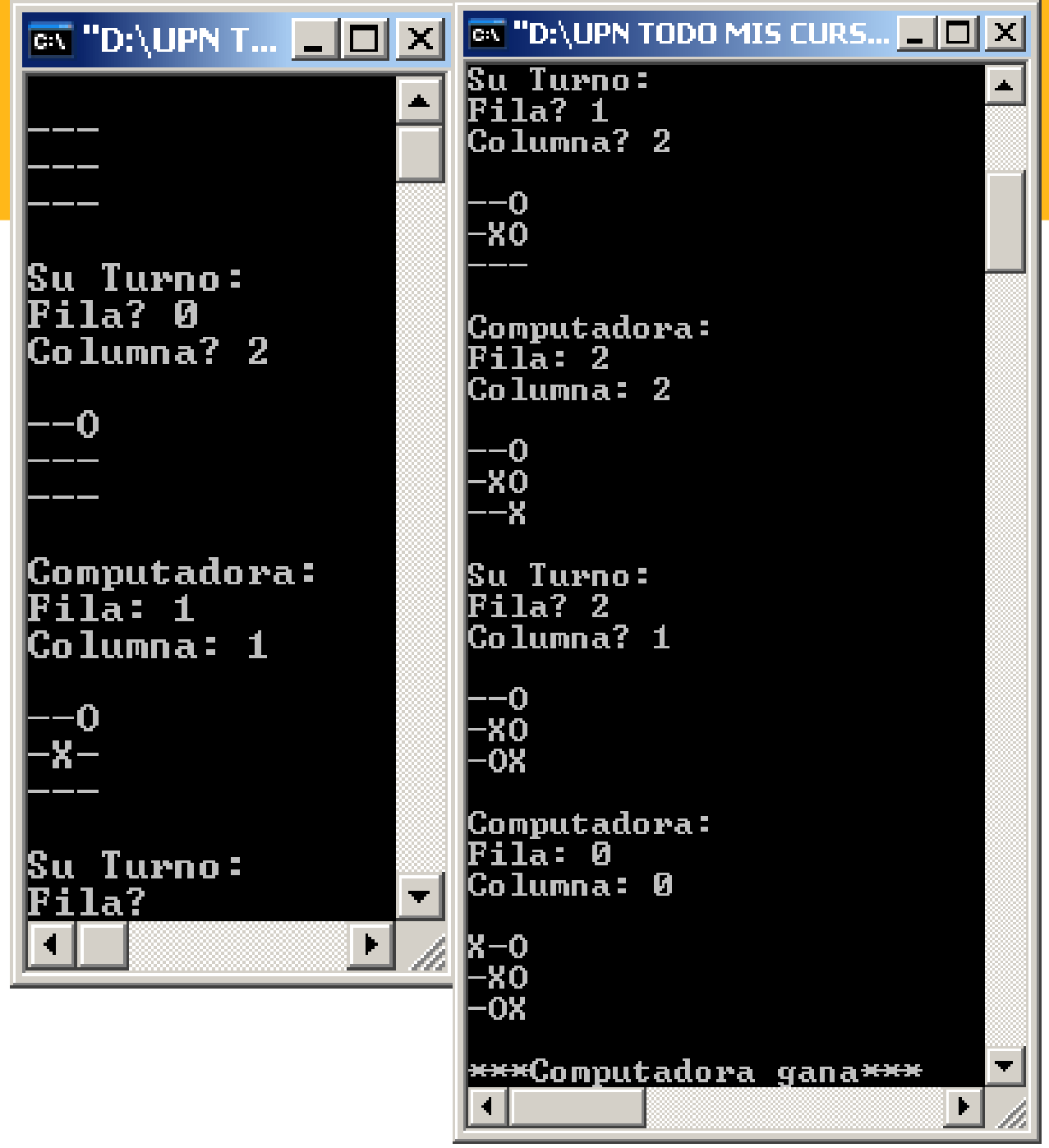




```
int eval(void)
{  int i, j, ganador=NADIE;
   // verifica filas
   for (i=0; i<3; i++)
       if( t[i][0]!= NADIE && t[i][0] == t[i][1] && t[i][1] == t[i][2] )
            ganador = t[i][0];
   // verifica columnas
   for (j=0; j<3; j++)
       if( t[0][j]!= NADIE && t[0][j] == t[1][j] && t[1][j] == t[2][j] )
            ganador = t[0][j];
   // verifica diagonal principal
   if( t[0][0]!= NADIE && t[0][0] == t[1][1] && t[1][1] == t[2][2] )
        ganador = t[0][0];
   // verifica diagonal secundaria
   if( t[0][2]!= NADIE && t[0][2] == t[1][1]  && t[1][1] == t[2][0]  )
                ganador = t[0][2];
   // verifica si hay ganador
   if( ganador==JUGADOR    ) return -1; // gana jugador
   if( ganador==COMPUTADOR ) return  1; // gana pc
   // mira si hay casillas vacias
   for (i=0; i<3; i++)
       for (j=0; j<3; j++)
            if (t[i][j]==NADIE)  return 2; // hay libres, aun nadie gana
   return 0;   // no hay libres, empate
}
```

```
void impTablero(void)
{
   int i, j;
   cout<<endl;
   for(i=0; i<3; i++)
   {
      for(j=0; j<3; j++)
           cout<<t[i][j];
      cout<<endl;
    }
   cout<<endl;
}
```




"D:\UPN T...

Su Turno:
Fila? 0
Columna? 2
--O

Computadora:
Fila: 1
Columna: 1
--O
-X-

Su Turno:
Fila?
"D:\UPN TODO MIS CURS...
Su Turno:
Fila? 1
Columna? 2
--O
-XO

Computadora:
Fila: 2
Columna: 2
--O
-XO
--X
Su Turno:
Fila? 2
Columna? 1
--O
-XO
-OX
Computadora:
Fila: 0
Columna: 0
X-O
-XO
-OX
Computadora gana

En el “3 en raya” entre humanos, es posible al menos empatar.

En nuestra implementación como recorremos todos los nodos, entonces no hay sorpresas, así que la computadora nunca perderá.

Puede modificar el programa para que solo recorra hasta una profundidad dada. Cuando llegue a esa profundidad evalúe el tablero según la siguiente función.

F(t) = ∑ filas que puede ganar MAXI
- ∑ filas que puede ganar MINNIE



F(t) = 2 – 3 = -1


Kahoot!

PRÓXIMAMENTE...

Programación dinámica



¡Gracias por su atención!

